

UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

"Plataforma web para la comercialización y revalorización de productos perecibles orientada al consumo sostenible en la provincia de Tacna"

Curso: Construcción de Software I

Docente: Mag. Ricardo Eduardo Valcárcel Alvarado

Integrantes:

Renzo Fernando Loyola Vilca ***(2021072615)***

Brant Antony Chata Choque ***(2020067577)***

Tacna – Perú

2026

I. INFORMACIÓN GENERAL

Campo	Detalle
Tema	Comparación entre Desarrollo de Software Basado en Componentes (DBC) y Desarrollo de Software Basado en Servicios (DBS)
Proyecto	ÓrbiKa — plataforma web para la comercialización y revalorización de productos perecibles en Tacna
Curso	Construcción de Software I
Docente	Mag. Ricardo Eduardo Valcárcel Alvarado
Integrantes	Brant Antony Chata Choque (2020067577) — Renzo Fernando Loyola Vilca (2021072615)
Repositorio	github.com/renzoloyola/G01_CSW_ORBIKA
Fecha	31 de octubre de 2026

II. MARCO TEÓRICO

Desarrollo de Software Basado en Componentes (DBC)

Organiza el sistema en unidades de software (componentes) con responsabilidades bien delimitadas, que exponen su funcionalidad mediante interfaces y se conectan entre sí por composición. El objetivo es favorecer la reutilización, la separación de responsabilidades y el mantenimiento independiente de cada unidad dentro de un mismo sistema.

Desarrollo de Software Basado en Servicios (DBS)

Organiza el sistema como un conjunto de servicios que exponen capacidades mediante contratos accesibles a través de una red (habitualmente HTTP/JSON), consumibles por distintos clientes —en este caso, una aplicación web React y, potencialmente, otros clientes autorizados—. El énfasis está en el contrato público (rutas, entradas, salidas, permisos) más que en la implementación interna.

Ambos enfoques no son excluyentes: en un mismo sistema, los componentes suelen organizar el código interno de un servicio, y los servicios exponen las capacidades de esos componentes hacia el exterior. Este informe aplica ambos enfoques al backend de ÓrbiKa: primero como una composición de componentes internos, y luego como los servicios REST que esos componentes exponen a la aplicación cliente.

III. PROCEDIMIENTO

1. Contexto de solución

ÓrbiKa es una plataforma web que conecta a vendedores de Tacna con excedentes o productos perecibles próximos a vencer, con compradores que buscan alimentos a menor precio. Centraliza publicación, búsqueda, compra y seguimiento, y busca reducir la merma de alimentos con resultados verificables. El sistema real está construido con React 18 (frontend), Node.js/Express (backend) y Supabase/PostgreSQL (persistencia y autenticación).

Código	Requerimiento	Función principal
RF-01	Registrar cuenta	Crear una cuenta de comprador o vendedor con correo único y campos obligatorios según el rol.
RF-02	Iniciar y cerrar sesión	Acceder con credenciales válidas y proteger las rutas privadas según el rol.
RF-07	Comprar / reservar producto	Confirmar un pedido de una cantidad disponible, reduciendo el stock de forma consistente.

2. Selección de componentes

Los componentes se seleccionan según su relación con los requerimientos, la posibilidad de reutilización y la separación de responsabilidades, siguiendo la organización real del código del repositorio.

Código	Componente	Responsabilidad	Implementación real
C01	Presentación React	Capturar acciones del usuario (formularios, catálogo, carrito de compra) y mostrar resultados.	Páginas y componentes en frontend/src/pages y frontend/src/components.
C02	AuthContext	Mantener la sesión del usuario y su perfil (rol, estado activo) en el cliente.	frontend/src/context/AuthContext.jsx.
C03	Gestión de productos	Validar y conservar publicaciones, calcular descuentos y disponibilidad.	backend/src/services/producto.service.js.
C04	Gestión de pedidos	Validar y coordinar la compra, cancelación, cambios de estado y calificación de pedidos.	backend/src/services/pedido.service.js.
C05	Autenticación y autorización	Verificar identidad (JWT de Supabase) y rol antes de operaciones protegidas.	backend/src/middlewares/auth.middleware.js (requireAuth, requireRole).
C06	Gestión de imágenes	Recibir fotografías de producto, validarlas y conservar su referencia.	backend/src/services/imagen.service.js.
C07	Acceso a datos	Ejecutar consultas y funciones transaccionales sobre PostgreSQL.	backend/src/config/supabaseClient.js y funciones RPC en supabase/migrations/.

3. Justificación de la selección

- Gestión de productos: concentra las reglas de publicación y descuento, evitando repetirlas entre el catálogo y la ficha de detalle.
- Gestión de pedidos: separa la lógica de compra, cancelación y calificación del resto del sistema, porque reúne la mayor cantidad de reglas de negocio e invariantes (stock, transiciones de estado).
- Autenticación y autorización: permite aplicar las mismas reglas de acceso (requireAuth, requireRole) a todas las operaciones protegidas, sin duplicarlas en cada ruta.
- Acceso a datos: evita distribuir consultas SQL entre controladores; las funciones transaccionales (RPC) concentran las invariantes críticas, como la reserva de stock.

- AuthContext: encapsula en el cliente el estado de sesión y evita que cada pantalla gestione el token de Supabase por su cuenta.

4. Composición de los componentes

La solución se organiza en tres grupos: presentación, lógica de aplicación y persistencia.

Grupo	Componentes	Forma de colaboración
Presentación	C01 y C02	Reciben acciones del usuario, mantienen la sesión y muestran resultados.
Lógica de aplicación	C03, C04, C05 y C06	Validan permisos, aplican reglas de negocio y coordinan operaciones.
Persistencia	C07	Conserva y recupera información mediante Supabase/PostgreSQL.

Flujo: registro e inicio de sesión (RF-01 / RF-02)

- La presentación React captura los datos del formulario de registro o inicio de sesión.
- AuthContext invoca directamente al SDK de Supabase Auth (signUp / signInWithPassword), sin pasar por el backend Express.
- Supabase valida el correo y la contraseña, crea la sesión y devuelve un token JWT.
- El backend (C05) solo entra en juego después: cada petición protegida posterior incluye ese token, que requireAuth valida contra Supabase antes de ejecutar la operación.

Flujo: compra con stock consistente (RF-07)

- La presentación React limita la cantidad seleccionable al stock disponible y genera una clave de idempotencia para el intento de compra.
- El backend (C05) verifica la autenticación y el rol "comprador".
- Gestión de pedidos (C04) valida los datos de entrada y delega la operación completa al acceso a datos (C07).
- Acceso a datos ejecuta la función transaccional crear_pedido_transaccion: bloquea la fila del producto, valida stock y vencimiento, crea el pedido y descuenta el stock en una sola transacción.
- El resultado (pedido confirmado o conflicto) regresa a través de las mismas capas hasta la presentación.

Diagrama de composición

El diagrama presenta los componentes de la solución y la dirección en que colaboran: la presentación invoca la lógica de aplicación, y esta accede a los datos a través de un único componente de acceso a datos.

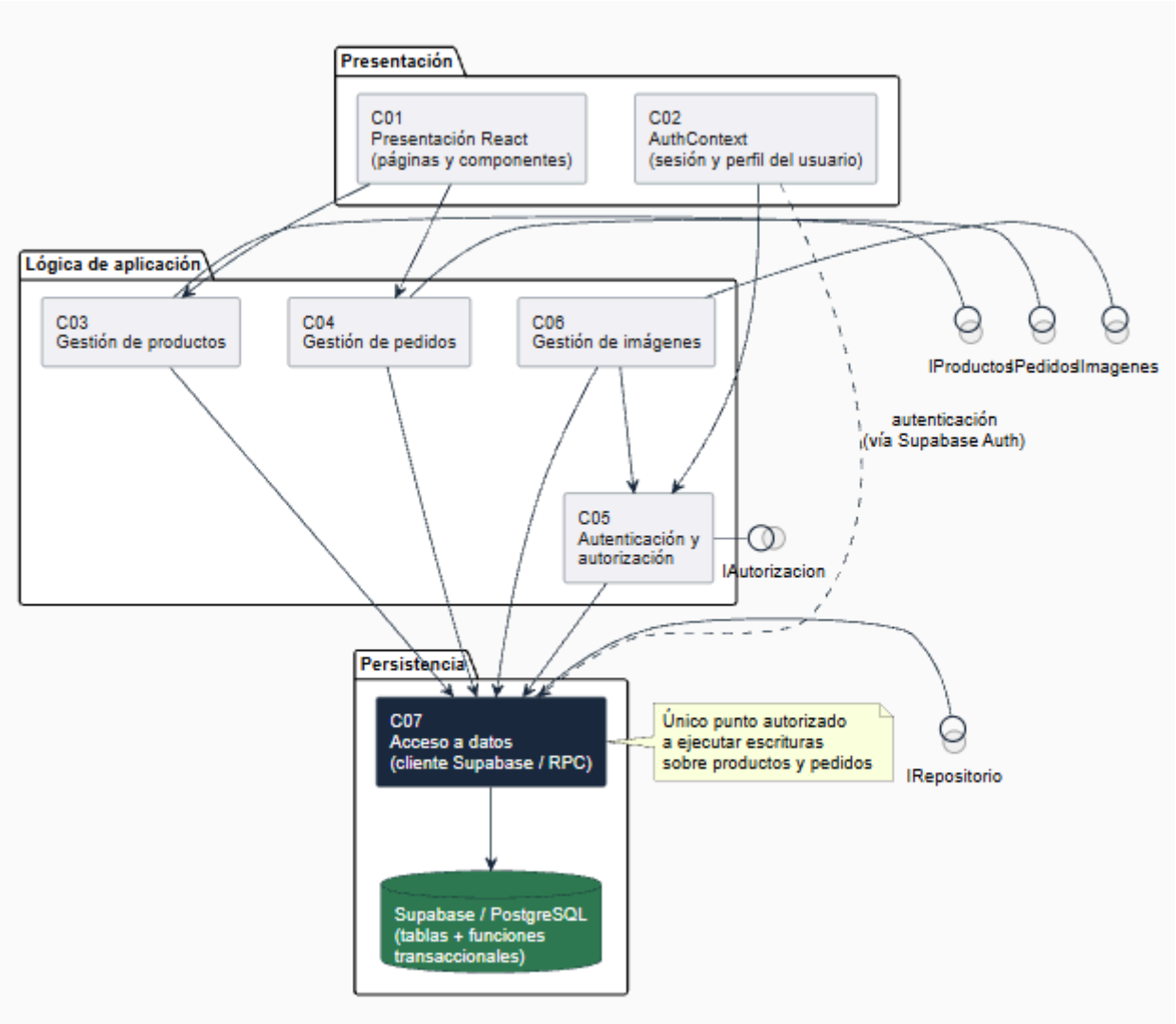


Figura 1. Composición de componentes de ÓrbiKa (presentación, lógica de aplicación y persistencia).

5. Configuración de interfaces

Una interfaz define qué operaciones ofrece un componente y qué condiciones deben respetar sus consumidores. Los nombres siguientes organizan el código real del backend.

Interfaz	Proporcionada por	Operaciones principales	Utilizada por
IProductos	C03	listarProductos(), obtenerProducto(), publicarProducto(), editarProducto()	Rutas públicas y privadas de productos.
IPedidos	C04	crearPedido(), listarPedidosDeUsuario(), cancelarPedido(), actualizarEstadoPedido(), calificarPedido()	Controlador de pedidos.
IAutorizacion	C05	requireAuth(), requireRole(...roles)	Todas las rutas protegidas del servidor.
IImagenes	C06	subirImagen()	Ruta de carga de fotografías de producto.

Interfaz	Proporcionada por	Operaciones principales	Utilizada por
IRepositorio	C07	Operaciones de Supabase (select/insert/update) y funciones RPC transaccionales.	Componentes C03–C06.

La interfaz de acceso a datos es la única autorizada a ejecutar operaciones de escritura sobre productos y pedidos: el cliente nunca escribe directamente en Supabase, todas las escrituras pasan por validación y autorización en el backend.

6. Contratos funcionales

Operación	Entrada	Salida	Condiciones
crearPedido	Actor autenticado, producto, cantidad, modalidad de entrega, referencia, clave de idempotencia.	Pedido confirmado (id, estado, monto) o error de conflicto.	Rol comprador; cuenta activa; cantidad ≤ stock disponible; producto no vencido.
publicarProducto	Actor autenticado, nombre, precio, stock, categoría, vencimiento, zona, modalidad.	Producto publicado con estado calculado.	Rol vendedor; vendedor activo; categoría activa; vencimiento futuro; stock > 0.
actualizarEstadoPedido	Pedido, actor, nuevo estado.	Pedido con estado actualizado.	Solo el vendedor avanza preparando→listo→entregado; solo el comprador confirma completado; transición válida según el estado actual.
calificarPedido	Pedido completado, puntaje (1–5), comentario opcional.	Calificación registrada.	Solo el comprador del pedido; pedido en estado "completado"; una calificación por pedido.

7. Configuración de conexiones

Conexiones entre componentes

Origen	Destino	Mecanismo	Configuración necesaria
Presentación React	AuthContext	Llamada directa a funciones del contexto.	Proveedor de contexto en el árbol de componentes.
AuthContext	Supabase Auth	SDK supabase-js en el cliente.	URL y clave pública (anon key) del proyecto Supabase.
Presentación React	API Express	Peticiones HTTP (axios) con token JWT.	URL base de la API (VITE_API_URL), CORS habilitado.
Controladores Express	Gestión de pedidos / productos	Invocación interna (import de funciones).	Ninguna adicional.

Origen	Destino	Mecanismo	Configuración necesaria
Gestión de pedidos / productos	Acceso a datos	Cliente supabase-js con clave de servicio (service role).	SUPABASE_URL y SUPABASE_SERVICE_ROLE_KEY en backend/.env.
Acceso a datos	PostgreSQL (Supabase)	Funciones RPC y operaciones REST de PostgREST.	Migraciones SQL aplicadas (001 a 004).

Condiciones de integración

- Seguridad: las operaciones protegidas se autorizan en el backend (requireAuth / requireRole), aunque la interfaz oculte opciones según el rol.
- Consistencia: la creación de un pedido y la reducción del stock se ejecutan dentro de una única función transaccional de PostgreSQL.
- Identidad: los reintentos de compra con la misma clave de idempotencia devuelven el pedido ya creado, sin duplicarlo.
- Separación de responsabilidades: las pantallas de React no ejecutan consultas SQL ni sustituyen las validaciones del servidor; el cliente solo guía al usuario (por ejemplo, limitando la cantidad al stock visible).
- Doble vía de acceso a datos: la autenticación se comunica directamente con Supabase desde el cliente, mientras que las operaciones de negocio (productos, pedidos, imágenes) siempre pasan por la API Express. Esta distinción se retoma en la sección de servicios.

8. Desarrollo de software basado en servicios

8.1 Necesidades de los servicios web

ÓrbiKa requiere comunicar la aplicación cliente (React, ejecutada en el navegador) con la base de datos central (PostgreSQL en Supabase). Para ello existe una API REST desarrollada en Express que aplica las reglas de negocio en el servidor, además del acceso directo del cliente al servicio gestionado de autenticación de Supabase.

Los servicios web permiten:

- Publicar y consultar productos del catálogo.
- Confirmar, cancelar y calificar pedidos, manteniendo el stock consistente.
- Recibir fotografías de producto y conservarlas.
- Verificar los permisos antes de modificar información.

La aplicación React y la API Express no acceden a PostgreSQL con las mismas credenciales: el cliente usa una clave pública (anon key) solo para autenticación; la API usa una clave de servicio (service role) para las operaciones de negocio.

8.2 Identificación de servicios

Código propuesto	Servicio	Requerimiento	Responsabilidad
SW-01	Servicio de productos	RF-03 / RF-19	Publicar, listar, obtener y editar productos del catálogo.
SW-02	Servicio de pedidos	RF-07 / RF-17 / RF-09	Confirmar, cancelar, calificar y listar pedidos con stock consistente.
SW-03	Servicio de imágenes	RF-07 (apoyo)	Recibir y validar fotografías de producto.
SW-04	Servicio de autenticación (gestionado)	RF-01 / RF-02 / RF-13	Registro, inicio de sesión, cierre de sesión y recuperación de contraseña, provisto por Supabase Auth y consumido directamente desde el cliente.

A diferencia de SW-01 a SW-03 —que son endpoints propios construidos en Express—, SW-04 es un servicio externo gestionado (Supabase Auth): el equipo no implementa su lógica, solo la consume con el SDK oficial desde React. Esta es una decisión arquitectónica real del proyecto, no una omisión: documentarla honestamente evita atribuir a un servicio propio una funcionalidad que en realidad provee un tercero.

8.3 Arquitectura de servicios

Las conexiones usan HTTP en desarrollo local y HTTPS en el entorno desplegado. Los datos se intercambian en JSON; las fotografías se transfieren como archivo binario con su tipo de contenido.

Elemento	Especificación real
Método y ruta	POST /api/pedidos
Consumidor	Presentación React (ProductDetail.jsx), a través de marketplace.comprarProducto().
Entrada	productoId, cantidad, modalidadEntrega, referenciaEntrega, claveIdempotencia; identidad tomada del token de sesión.
Identidad	Se obtiene validando el encabezado Authorization (Bearer) contra Supabase Auth; se contrasta con el perfil y su rol.
Resultado	Pedido confirmado (201) o pedido ya existente con la misma clave reconocido sin duplicarse.
Validaciones	Rol comprador, cuenta activa, cantidad entera positiva, modalidad de entrega válida, producto con stock suficiente.

Diagrama de arquitectura de servicios

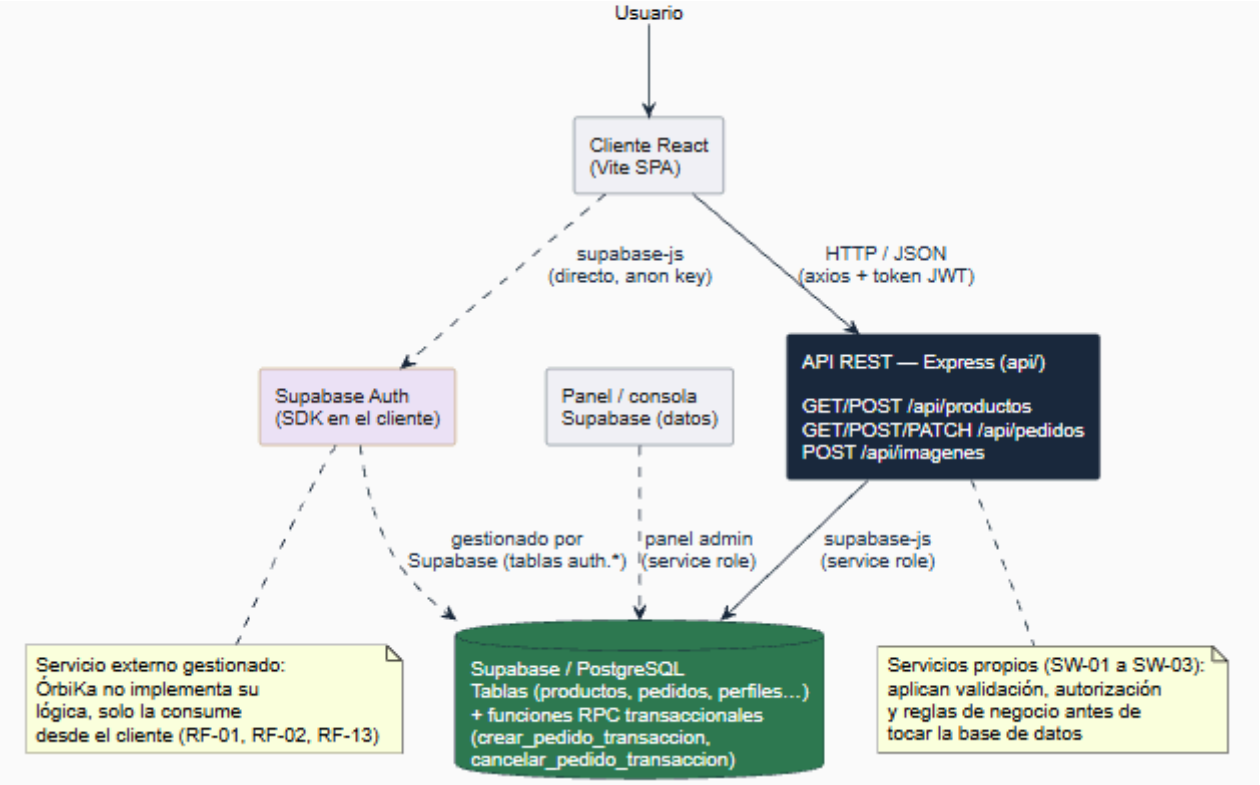


Figura 2. Arquitectura de servicios de ÓrbiKa: API REST propia (Express) y servicio gestionado de autenticación (Supabase Auth), ambos sobre la misma base PostgreSQL.

8.4 Contrato de los servicios

SW-01 Productos

Método y ruta (relativa a /api)	Operación y permiso
GET /productos	Listar y filtrar productos publicados. Público.
GET /productos/:id	Obtener el detalle de un producto. Público.
POST /productos	Publicar un producto nuevo. Vendedor autenticado.
PUT /productos/:id	Editar un producto propio. Vendedor autenticado, dueño del producto.

SW-02 Pedidos

Método y ruta (relativa a /api)	Operación y permiso
POST /pedidos	Confirmar un pedido. Comprador autenticado.
GET /pedidos	Listar los pedidos propios (como comprador o vendedor). Autenticado.
PATCH /pedidos/:id/cancelar	Cancelar un pedido propio aún cancelable. Comprador.
PATCH /pedidos/:id/estado	Avanzar el estado del pedido. Comprador o vendedor, según la transición.
POST /pedidos/:id/calificacion	Calificar un pedido completado. Comprador, una vez.

Los reintentos de POST /pedidos con la misma clave de idempotencia no duplican el pedido; si los datos cambian con la misma clave, el servidor responde con un conflicto (409) en lugar de crear o modificar silenciosamente el registro existente.

SW-03 Imágenes

Método y ruta (relativa a /api)	Operación y permiso
POST /imagenes	Subir una fotografía de producto (JPEG/PNG, máx. 5 MB). Vendedor autenticado.

8.5 Creación de servicios en Express

Elemento	Función	Archivo real
Rutas	Declarar el método HTTP, la ruta y el middleware de permisos.	backend/src/routes/*.routes.js
Controlador	Recibir la petición, invocar el servicio y devolver la respuesta HTTP.	backend/src/controllers/*.controller.js
Validación	Comprobar estructura, campos y tipos de datos.	backend/src/validation/*.validation.js
Autorización	Verificar identidad, rol y, cuando aplica, titularidad del recurso.	backend/src/middlewares/auth.middlewar e.js
Lógica de negocio	Aplicar las reglas del requerimiento.	backend/src/services/*.service.js
Acceso a datos	Ejecutar operaciones o funciones RPC sobre PostgreSQL.	backend/src/config/supabaseClient.js + supabase/migrations/*.sql

Para confirmar un pedido, el flujo real es:

- La ruta POST /api/pedidos aplica requireAuth y requireRole("comprador").
- El controlador invoca pedidoService.crearPedido(req.perfil, req.body).
- El servicio valida los datos de entrada con validarCompra() y rechaza cuentas inhabilitadas.
- El servicio delega la operación a la función transaccional crear_pedido_transaccion vía RPC.
- El controlador responde 201 con el pedido, o propaga el error (409 por conflicto de stock, 403 por rol, etc.).

8.6 Consumo desde la interfaz React

En ProductDetail.jsx, el usuario elige cantidad y modalidad de entrega; el formulario limita la cantidad al stock visible y genera una clave de idempotencia (crypto.randomUUID()) antes de mostrar el modal de confirmación. Al confirmar, marketplace.comprarProducto() envía la petición a la API y, si la respuesta es exitosa, la interfaz actualiza el stock local y muestra el número de pedido; si falla, muestra el mensaje de error devuelto por el servidor sin asumir que el pedido se creó.

Para la autenticación, AuthContext.jsx llama directamente a supabase.auth.signUp() o signInWithPassword(); no existe una ruta propia de ÓrbiKa para estas operaciones. El backend solo entra en juego para validar el token en peticiones posteriores.

9. Manual técnico de la solución

9.1 Objetivo

Describir la organización, configuración y funcionamiento reales de ÓrbiKa, para facilitar su instalación, prueba y mantenimiento.

9.2 Herramientas utilizadas

Tecnología o herramienta	Uso
React 18 + Vite	Aplicación cliente (SPA).
Node.js + Express	API REST y reglas de negocio del servidor.
Supabase (PostgreSQL + Auth)	Persistencia central, funciones transaccionales y autenticación gestionada.
Vitest	Pruebas automatizadas de backend y frontend.
PGlite (@electric-sql/pglite)	Base PostgreSQL real en memoria para pruebas de integración de las migraciones SQL.
Axios	Cliente HTTP del frontend hacia la API.

9.3 Organización del repositorio

Directorio	Contenido
backend/src/routes/	Declaración de rutas y middlewares de permisos.
backend/src/controllers/	Recepción de peticiones y respuestas HTTP.
backend/src/services/	Reglas de negocio de productos, pedidos e imágenes.
backend/src/validation/	Validación de entradas por módulo.
backend/src/middlewares/	Autenticación y autorización (requireAuth, requireRole).
backend/src/config/	Cliente de Supabase (service role).
supabase/migrations/	Esquema, políticas de seguridad y funciones transaccionales.
frontend/src/pages/	Pantallas (catálogo, detalle, autenticación, mis pedidos…).
frontend/src/context/	AuthContext y estado de sesión.
frontend/src/services/	Cliente HTTP (marketplace.js) y cliente de Supabase.

9.4 Instalación y configuración

- Clonar el repositorio y crear un proyecto en Supabase.
- Aplicar en orden las migraciones de supabase/migrations/ sobre ese proyecto.
- Copiar backend/.env.example a backend/.env y completar SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY y FRONTEND_URL.
- Ejecutar npm install y npm run dev dentro de backend/.
- Configurar la URL de la API y la clave pública de Supabase en el frontend.
- Ejecutar npm install y npm run dev dentro de frontend/.
- Preparar usuarios de prueba con roles diferenciados (comprador, vendedor).

- Comprobar la comunicación entre React, la API y Supabase antes de continuar.

9.5 Configuración de acceso a datos

El backend centraliza el acceso a PostgreSQL mediante supabase-js, con una clave de servicio que nunca se expone al cliente. Las operaciones que modifican stock o estado de pedidos se implementan como funciones de PostgreSQL (plpgsql), no como varias instrucciones sueltas desde Node.js, para que la reducción de stock y la creación del pedido ocurran como una sola unidad atómica, con bloqueo de fila sobre el producto mientras se evalúa el stock disponible.

9.6 Respuestas y tratamiento de errores

Código	Significado real en el proyecto	Origen
200 / 201	Operación correcta / recurso creado.	Respuesta normal del controlador.
400	Entrada inválida (VALIDATION_ERROR).	Capa de validación.
401	Sin sesión o token inválido.	requireAuth.
403	Rol insuficiente o recurso ajeno (FORBIDDEN).	requireRole / reglas del servicio.
404	Recurso no encontrado (ORDER_NOT_FOUND).	Servicio de pedidos.
409	Conflicto de stock, estado o datos (DATA_CONFLICT / INVALID_TRANSITION).	Función transaccional / servicio.
502	Respuesta inesperada de persistencia (ORDER_CREATE_FAILED).	Servicio de pedidos.

9.7 Mantenimiento

Se deben mantener copias de seguridad de la base de datos, registrar errores sin exponer credenciales ni datos personales, documentar cada nueva migración SQL y comprobar la compatibilidad entre el esquema de Supabase y el código del backend antes de desplegar cambios.

10. Aseguramiento de calidad

10.1 Verificación

La verificación comprueba que la solución se construya conforme a sus especificaciones.

Actividad	Aplicación al proyecto
Revisión de contratos	Comprobar entradas, respuestas y códigos de error de la API de productos y pedidos.
Revisión de código	Verificar separación de responsabilidades entre rutas, controladores, servicios y acceso a datos.
Pruebas unitarias	Evaluar reglas de validación de pedidos, productos y autenticación de forma aislada (mocks).
Pruebas de integración	Comprobar las funciones transaccionales contra una base PostgreSQL real (PGlite).
Pruebas de permisos	Rechazar operaciones fuera del rol o sin sesión (401 / 403).

Actividad	Aplicación al proyecto
Pruebas de persistencia	Comprobar que el stock y el pedido se confirman juntos, sin registros parciales.

10.2 Validación

La validación comprueba que la solución permita realizar las tareas que necesitan los usuarios.

Participante	Tarea de validación
Comprador	Registrarse, iniciar sesión y confirmar la compra de un producto disponible.
Vendedor	Publicar un producto y revisar los pedidos recibidos.
Usuario sin sesión	Comprobar que las rutas protegidas no quedan accesibles.
Equipo de desarrollo	Confirmar que un reintento de compra no genera pedidos duplicados.

10.3 Casos de prueba y resultados reales

A diferencia de un plan de pruebas puramente propuesto, los siguientes casos ya cuentan con una ejecución real documentada, obtenida al correr las suites de Vitest del proyecto (backend con PGlite y mocks; frontend) durante la elaboración de este informe y del informe de trazabilidad del Sprint 4.

ID	Requisito	Prueba y resultado esperado	Resultado
CP-01	RF-01	Registrar una contraseña que no cumple la robustez exigida o no coincide con su confirmación es rechazado.	**PASS — evidencia real**
CP-02	RF-01	El registro de una cuenta vendedor exige nombre de negocio y ubicación; el de comprador no.	**PASS — evidencia real**
CP-03	RF-13 (apoyo a RF-01)	Se genera correctamente la ruta de retorno para restablecer contraseña.	**PASS — evidencia real**
CP-04	RF-01	Registrar una cuenta nueva directamente contra un proyecto Supabase real (signUp).	**Pendiente de ejecución**
CP-05	RF-02	Acceder a una ruta protegida (crear pedido) sin sesión iniciada es rechazado con 401.	**PASS — evidencia real**
CP-06	RF-02	Un comprador autenticado no puede publicar un producto (rol incorrecto, 403).	**PASS — evidencia real**
CP-07	RF-07	Comprar una cantidad disponible crea el pedido y reduce el stock exactamente una vez.	**PASS — evidencia real**
CP-08	RF-07	Una confirmación repetida con la misma clave de idempotencia no duplica el pedido.	**PASS — evidencia real**
CP-09	RF-07	Una compra con cantidad mayor al stock disponible es rechazada y el stock no cambia.	**Pendiente de ejecución**
CP-10	RF-07	Dos solicitudes de compra simultáneas sobre las mismas unidades no dejan el stock en negativo.	**Pendiente de ejecución**

Resumen: 7 de 10 casos cuentan con evidencia real de ejecución (PASS); 3 permanecen pendientes porque requieren, respectivamente, un proyecto Supabase real conectado y dos pruebas de base de

datos aún no escritas (compra por encima del stock y compra concurrente). Estos dos últimos son el mismo hallazgo documentado en el informe de trazabilidad del Sprint 4.

11. Comparación aplicada al proyecto

Aspecto	Basado en componentes	Basado en servicios
Propósito	Organizar la implementación interna en unidades con responsabilidades específicas.	Ofrecer capacidades mediante contratos HTTP accesibles a consumidores.
Unidad principal	Componente de productos, pedidos, autenticación o acceso a datos.	Servicio de productos, de pedidos, de imágenes y de autenticación (gestionado).
Contrato	Funciones e interfaces internas entre módulos de Node.js.	Rutas, métodos HTTP, cuerpos JSON y códigos de estado.
Comunicación	Llamadas de función dentro del mismo proceso (import).	Peticiones HTTP/JSON y, para autenticación, el SDK de Supabase.
Ejemplo real	pedido.controller.js invoca pedidoService.crearPedido().	React invoca POST /api/pedidos; AuthContext invoca supabase.auth.signUp().
Reutilización	Compartir validación, autorización y acceso a datos entre servicios de negocio.	Consumir el mismo endpoint desde distintas pantallas o, en el futuro, otros clientes autorizados.
Integración	Composición mediante interfaces internas (IPedidos, IRepositorio…).	Coordinación entre el cliente React y la API, y entre el cliente y Supabase Auth directamente.
Fallos principales	Errores de lógica, validación interna o persistencia.	Además, fallos de red, tiempos de espera y respuestas perdidas.
Pruebas	Unitarias de servicios con mocks (Vitest); integración contra PostgreSQL real (PGlite).	Pruebas HTTP de extremo a extremo (test/http/app.test.js) y de permisos por ruta.
Aplicación conjunta	Los componentes implementan las reglas de negocio dentro del backend.	Los servicios exponen esas reglas a la aplicación React y, para autenticación, delegan en Supabase.

En ÓrbiKa, ambos enfoques no son alternativas sino capas complementarias: los componentes (C03–C07) organizan el código interno del backend, y ese mismo backend se expone como servicios (SW-01 a SW-03) que React consume. La particularidad del proyecto es que no todos los servicios son propios: la autenticación (SW-04) es un servicio externo gestionado por Supabase, consumido directamente desde el cliente sin pasar por los componentes del backend. Esta combinación —componentes propios + servicios propios + un servicio gestionado de terceros— refleja una arquitectura real basada en BaaS (Backend as a Service), más que un backend monolítico tradicional.

12. Conclusiones

- El desarrollo basado en componentes permitió organizar el backend de ÓrbiKa en módulos de productos, pedidos, autenticación, imágenes y acceso a datos. La definición de interfaces

(IProductos, IPedidos, IRepositorio…) delimita sus responsabilidades y concentra las invariantes críticas —como la reserva de stock— en un único punto del sistema.

- El enfoque basado en servicios permite comunicar la aplicación React con la API Express mediante contratos HTTP claros (rutas, entradas, salidas, permisos), y evidenció una decisión arquitectónica real del proyecto: la autenticación no es un servicio propio, sino un servicio gestionado (Supabase Auth) consumido directamente desde el cliente.
- La verificación con pruebas reales confirma que 7 de los 10 casos propuestos ya tienen evidencia de ejecución (registro, permisos por rol, compra con reducción de stock e idempotencia); los 3 casos pendientes —registro contra un Supabase real, compra que excede el stock y compra concurrente— quedan señalados explícitamente, no dados por aprobados sin evidencia.
- La comparación aplicada al proyecto muestra que ambos enfoques se complementan: sin componentes bien delimitados, los servicios expondrían lógica desordenada; sin un contrato de servicios claro, la aplicación React no podría integrarse de forma predecible con el backend ni con Supabase.

13. Recomendaciones

- Mantener actualizados los contratos de la API (rutas, entradas, respuestas, permisos) a medida que se agreguen nuevos servicios, para conservar la compatibilidad entre React y Express.
- Completar las pruebas pendientes de RF-07 (compra que excede el stock y compra concurrente) antes de dar por cerrado el Definition of Done del requisito, tal como se señaló en el informe de trazabilidad del Sprint 4.
- Documentar explícitamente, en el SAD, la distinción entre servicios propios (Express) y el servicio gestionado de autenticación (Supabase Auth), para que la arquitectura descrita coincida con la implementada.
- Configurar variables de entorno de prueba (sin credenciales reales) para que las suites que dependen de Supabase se ejecuten también en integración continua, evitando que queden sin cargar como ocurre actualmente sin un archivo .env.

14. Bibliografía y fuentes técnicas

- Grupo G01. (2026). Especificación de requisitos de software (SRS) de ÓrbiKa, RF-01, RF-02 y RF-07.
- Grupo G01. (2026). Planificación de Sprints de ÓrbiKa, versión 1.0, 12 de octubre de 2026.
- Grupo G01. (2026). Informe de Trazabilidad — Sprint 4 ÓrbiKa (evidencia de pruebas reales reutilizada en este informe).
- Repositorio ORBIKA. (2026). backend/src/services/pedido.service.js, producto.service.js, imagen.service.js.
- Repositorio ORBIKA. (2026). backend/src/middlewares/auth.middleware.js, backend/src/routes/*.routes.js.
- Repositorio ORBIKA. (2026). supabase/migrations/002_security_and_order_transactions.sql y 003_srs_integrity_and_images.sql.
- Repositorio ORBIKA. (2026). frontend/src/context/AuthContext.jsx, frontend/src/pages/ProductDetail.jsx, frontend/src/services/marketplace.js.
- Repositorio ORBIKA. (2026). backend/test/http/app.test.js, backend/test/sql/database.test.js, frontend/src/context/auth.helpers.test.js.

Repositorio: https://github.com/renzoloyola/G01_CSW_ORBIKA